队伍编号	MCB2302202
赛道	A

基于深度学习的坑洼道路检测和识别研究

摘 要

道路的状况对于交通安全至关重要。坑洼道路是指路面出现了凹凸不平、有大大小小的裂缝和洞坑等问题，不仅会给车辆和行人带来不便，还可能导致交通事故的发生。随着近年来科技的进步，深度学习技术迅速发展，为坑洼道路检测任务提供了新的解决方案。本文结合所给的图像文件，提取了图像特征，建立了识别图像道路坑洼与否的模型，并从不同维度进行模型评估，并利用该模型对测试集中的图像进行分类识别。

针对问题一，需要建立一个识别道路坑洼的模型，由于数据集中 normal 和 potholes 类的样本数量不平衡，所以对 potholes 类数据进行**翻转**和**四等分裁剪**操作以达到**平衡数据集**的效果。在图像特征提取方面，本文使用 **VGG16 获取图片特征向量**并使用 **PCA 降维**，同时分析图像颜色分布直方图和使用 **Canny 算法检测轮廓特征**。为了提高模型准确率，本文引入两个深度学习模型 **Yolov5** 和 **Yolov8**，同时加入一个基于集成学习的机器学习模型。并采用**加权投票**的方法将**机器学习、Yolov5** 和 **Yolov8** 的结果进行集成，权重分别为 0.2、0.4、0.4。这样可以有效解决两个深度学习模型分类结果歧义问题。

针对问题二，对模型进行了完整的训练和评估，从**指标评价**、**鲁棒性**和**生成对抗样本**三个方面进行了分析。分别考察了**损失函数变化**、**精确度**和**召回率**、**高斯噪声鲁棒性**、**生成对抗样本**及识别等，得出了模型具有一定的鲁棒性和抗对抗样本能力的结论。

对于 Yolov5 模型，其各项**损失函数随着迭代的增加而逐渐降低、精确度和召回率整体呈上升趋势**，具体见图 22；在噪声数据上，有 4 个样本前后识别结果不一致，**分类准确率为 97.010%，鲁棒性较强**；不考虑对抗攻击强度因素，在对抗样本上，**仅有 1 个对抗样本前后识别结果不同，抗对抗样本能力较强**。

对于 Yolov8 模型，其**损失函数同样随着迭代周期增加而降低**，具体见图 24，**F1** 在置信度为 0.453 时达到**最大值 0.59**，**精确度**随置信度增加呈上升趋势，**在 0.914 置信度达到 1.0**；在噪声数据上，9 个样本前后识别不一致，分类准确率下降了 2.325%，**为 96.678%，具有一定鲁棒性**；不考虑对抗攻击强度因素，在对抗样本上，**仅有 2 个对抗样本前后识别结果不同，抗对抗样本能力较强**。

综合上述评估指标，**本文所构建的模型识别率较高、分类准确、鲁棒性较好，且具有一定的抗对抗样本的能力**。

针对问题三，将训练好的模型用于识别所给训练集中，对于正常道路需要标记为 1，对于坑洼道路需要标记为 0，并将结果存入到“test_resulu.csv”文件中。

关键词：Yolov5　Yolov8　集成学习　VGG16　Canny 算法　生成对抗样本

目录

一、 问题重述 1
1.1 问题背景 1
1.2 问题提出 1
二、 问题分析 1
2.1 问题一的分析 1
2.2 问题二的分析 2
2.3 问题三的分析 2
三、 符号说明 2
四、 模型建立与求解 2
4.1 问题一的模型建立与求解 2
4.1.1 数据预处理 3
4.1.2 图像特征提取 4
4.1.3 模型建立 7
4.1.4 基于集成学习的机器学习模型 8
4.1.5 基于 Yolov5 的深度学习模型 9
4.1.6 基于 Yolov8 的深度学习模型 13
4.2 问题二的模型建立与求解 16
4.2.1 模型训练 16
4.2.2 模型的指标评价 16
4.2.3 模型的鲁棒性 19
4.2.4 添加高斯噪声 19
4.2.5 生成对抗样本 20
4.3 问题三的模型建立与求解 23
五、 模型评价与推广 24
5.1 模型优点 24
5.2 模型缺点 24
5.3 模型推广 24
六、 参考文献 24

一、 问题重述

1.1 问题背景

基于计算机视觉的坑洼道路检测和识别是一个具有实际应用意义的问题。及时检测和识别坑洼道路可以帮助城市和交通管理部门快速定位和修复道路问题，从而提升道路的安全性和可行性。通过减少坑洼道路的存在，可以减少车辆事故和损坏，并提高驾驶者和乘客的舒适度。对于自动驾驶车辆而言，准确检测和识别坑洼道路至关重要。自动驾驶车辆需要实时感知道路状况并作出相应的决策，以确保安全行驶。通过坑洼道路的检测和识别，自动驾驶车辆可以调整车速、避免碰撞或选择更平整的道路。除了日常生活，坑洼道路检测和识别在地质勘探、航天科学和自然灾害等领域的研究和应用具有重要意义。

随着近年来科技的进步，深度学习技术迅速发展，为坑洼道路检测任务提供了新的解决方案。在识别坑洼的过程中，深度学习可以提取坑洼的轮廓、纹理和形态等特征，并将其转化为更易于分类的表示形式。例如，一些研究者采用基于深度学习的方法对道路图像进行分类，将其分为正常和坑洼两类。他们利用深度学习模型从大量带有标签的训练图像中学习坑洼图像的特征，并通过训练模型进行分类预测。此外，一些研究者利用迁移学习的思想，利用在大规模图像数据上预训练的通用模型（如 ImageNet 上的预训练模型），学习坑洼图像的特征表示。他们通过微调或仅训练分类层，将预训练模型的知识迁移到坑洼图像分类任务中，以提高分类性能。

总而言之，深度学习技术在坑洼道路检测任务中具有重要作用。它能够自动提取关键特征，并利用迁移学习等方法提升分类准确性。这些方法为解决坑洼道路检测问题提供了更可靠和有效的解决方案。

1.2 问题提出

本次赛题提供了道路图像数据，对此进行分析、特征提取和建模，根据建立的模型自动识别一张新的道路图像的坑洼状态。基于如上要求，完成下列任务：

问题一：利用所提供道路图像数据，提取图像特征，建立一个识别图像道路是正常还是坑洼的模型，要求识别率高、速度快并且分类准确。

问题二：训练问题一中所构建的用于识别道路坑洼与否的模型，并从不同维度评估该模型。

问题三：利用上述问题中构建并训练的模型判断测试集中的道路图像是正常或者坑洼，并将识别结果保存至“test_result_csv”文件中。

二、 问题分析

2.1 问题一的分析

问题一要求结合所给的图像文件，提取图像特征，在此基础上建立一个识别道路坑洼与否且性能好的模型。通过分析发现，所给的图像文件中 normal 类和 potholes 类的样本数量相差较大，因此需要在原来 potholes 类数据集上进行翻转和四等分裁剪操作来扩充 potholes 类数据集，以到达平衡数据集的效果。

在图像特征提取方面，1)使用深度学习 VGG16 获取图片的特征向量并使用主成分分析法进行降维；2)分析图像的颜色分布直方图；3)使用 Canny 算法检测图像轮廓特征。

在模型建立方面，为了提高模型的识别率和分类准确度我们引入两个深度学习模型 Yolov5 和 Yolov8，但由于这两个模型的分类会存在歧义问题，所以我们使用集成学习的方法并引入机器学习。通过加权投票的方法，将机器学习、Yolov5、Yolov8 的权重分别设置为 0.2、0.4、0.4，这样就可以有效解决两个深度学习模型分类结果歧义问题，即当深度学习模型分类结果相同时，采用深度学习结果；当深度学习模分类结果不同时，采用机器学习结果。

2.2 问题二的分析

问题二中首先要完整对模型的训练，接着从不同的维度对模型进行评估。在评估上我们从指标评价、鲁棒性、生成对抗样本三个维度进行了分析。在指标评价上，我们展示了各损失函数随着训练周期增长的变化规律，以及模型的精确度和召回率；在模型鲁棒性上，在原始数据集中添加高斯噪声，并将原分类结果和加入噪声后的分类结果进行对比，从而评估模型的鲁棒行；在生成对抗样本上，采用遗传算法生成 35 张有坑洼图片在 Yolov5 和 Yolovv8 上的对抗样本，并将对抗样本进行识别，找到前后变化的对抗样本，如果样本数量不多，则认为具有抗对抗样本的能力，有一定的安全性。

2.3 问题三的分析

在问题三中，需要我们利用已经训练好的模型去识别训练集中的坑洼图像，并将结果存入到“test_resulu.csv”文件中，对于正常道路需要标记为 1，对于坑洼道路需要标记为 0。

三、 符号说明

名称	解释
X	图像特征向量
$f(\cdot)$	优化目标函数
$F(\cdot)$	图像识别模型
c	权衡攻击成和最终图像的权重系数
l_{model}	与识别模型相关的损失函数
$l_{metric}(x+\delta)$	模型对样本$x+\delta$的置信度

四、 模型建立与求解

4.1 问题一的模型建立与求解

在问题一中面临两个关键任务：首先，需进行图像特征提取以区分正常类和坑洼类两个类别；其次，需要构建一个专用模型，以便对图像进行分类，以确定道路状态是否正常或存在坑洼。

4.1.1 数据预处理

(1) 平衡数据集

当目标检测模型面临类别不平衡的数据集时，存在过度拟合样本更多类别、忽视小样本类别的问题。这种情况下，模型往往倾向于偏向于预测数量较多的类别，而对于数量较少的类别则表现出相对较低的关注度和准确性。数据扩增成为了该领域中一种常用的解决方案。在本次赛题中，我们发现其提供的道路图像数据集存在 normal 类 266 张、potholes 类 35 张的不平衡情况，为此，我们对 potholes 类数据集进行了扩增。具体而言，我们通过将 35 张 potholes 类道路图像进行翻转，并进行四等分裁剪的方式，得到了更多的数据集。这一方法能够有效地缓解模型对数量较多类别的偏向，提高对数量较少类别的关注度和准确性。



图 1 翻转操作图



图 2 裁剪操作图

对图片进行预处理的目的是增加 potholes 类的数据集，因此裁剪的过程中只保留有坑洼的图片，**如图 2 中排除了无坑洼的图片**。

(2) 标识坑洼

为了应对本次赛题中的坑洼图像识别任务，我们采用了 labelimg 工具，通过给 potholes 类添加 potholes 标记，以便用于 Yolo 模型的训练。

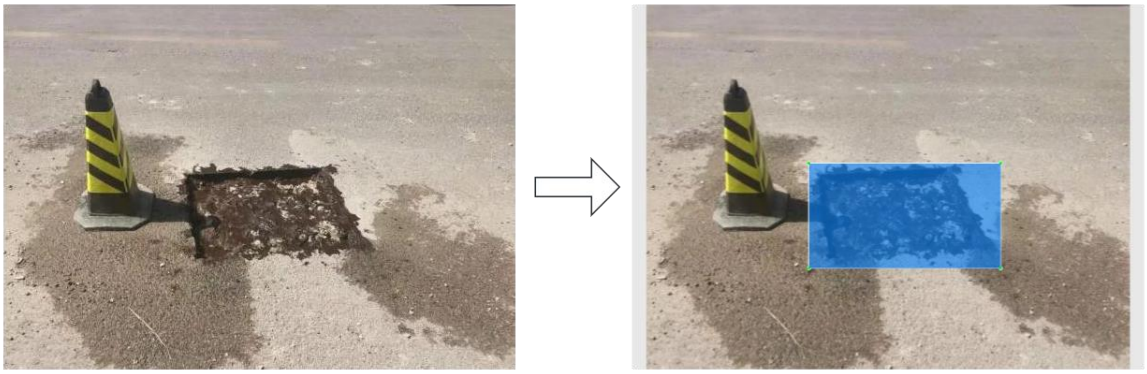


图 3 标识坑洼图

4.1.2 图像特征提取

在提取图像特征方面我们进行了三个方面的分析：

1、使用深度学习 VGG16 得到每张图片的特征向量，接着使用主成分分析法将其降维到一个长度为 100 的特征向量$X_1=(x_{1_1},x_{1_2},\cdots,x_{1_{100}})$；

2、分析图像的颜色分布直方图得到图像 RGB 上三个维度上长度为 256 的特征向量，即$X_i=(x_{i_1},x_{i_2},\cdots,x_{i_{256}}),i=2,3,4$；

3、首先使用 Canny 边缘检测算法找到有用的边缘，接着在 Canny 检测到的边缘中查找封闭的轮廓，最后计算出封闭轮廓面积占该图片面积的比例，得到一个一维特征向量$X_5=(x_5)$。

最后利用这三个方面得到的特征向量，接着便可以构建出每张图片对应的特征向量$X=(X_1,X_2,X_3,X_4,X_5)=(X_{1_1},\cdots,X_{1_{100}},X_{2_1},\cdots,X_{2_{256}},X_{3_1},\cdots,X_{3_{256}},X_{4_1},\cdots,X_{4_{256}},X_{5_1})$。并利用该特征向量来区分道路是正常还是坑洼。

（一）深度学习模型提取图像特征

VGG16 模型在 2014 年的 ImageNet 图像分类与定位挑战赛中确实取得了显著的成绩，展现了其出色的性能。其主要优点之一是其结构的简洁性和深度。这种深度使得模型能够捕捉到图像中更复杂和更高级的特征。

VGG16 特意使用小的3×3卷积核，这与许多其他模型使用的较大卷积核不同。小的卷积核有几个优点。首先，它们减少了参数数量，这有助于防止过拟合，同时保持了模型的计算效率。其次，使用多个小卷积核的连续卷积层可以模拟较大卷积核的效果，但是带有更少的参数和更高的非线性，这使得模型可以学习更复杂的特征。

此外，VGG16 的设计也确保了每一个卷积层的输出与前一个卷积层具有相同的空间维度（宽和高），这意味着仅当达到池化层时，特征图的尺寸才会减半。使用2×2大小、步幅为 2 的 max 池化方式可以确保每一层的输出尺寸是前一层的一半。

VGG16 由若干卷积层和池化层按顺序堆叠而成，通过这种方式，VGG16 能够形成一个深度网络结构，这对于捕捉图像中的细微和高级特征至关重要。

当使用 VGG16 进行特征提取时，并不需要得到类别的概率分布，所以此处我们移除了最后的 Softmax 激活函数。修改后 VGG16 进行特征提取的具体结构图如下：

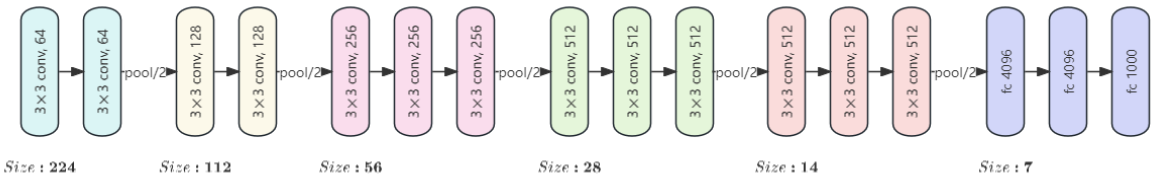


图 4 用于特征提取的 VGG16 结构图

使用 VGG16 进行完特征提取后得到了长度为 1000 的特征向量，然而高维数据会导致更高的计算成本，因此选择使用主成分分析法对该特征向量进行降维，最后得到长度为 100 的特征向量 $X_1=(x_{1_1},x_{1_2},\cdots,x_{1_{100}})$。

（二）颜色分布直方图

颜色分布直方图是一种用于描述图像中颜色分布的重要特征。通过研究直方图的形状和峰值可以反映出主要色调分布和对比度情况。

在绘制颜色分布直方图时，首先，计算出每张图片在 RGB 三个通道上的像素分布情况；接着，按 normal 类中的 R 通道、normal 类中的 G 通道、normal 类中的 B 通道、potholes 类中的 R 通道、potholes 类中的 G 通道、potholes 类中的 B 通道进行分类；最后，将每个类中的分布进行累加并绘制出直方图。得到的颜色分布直方图如下：

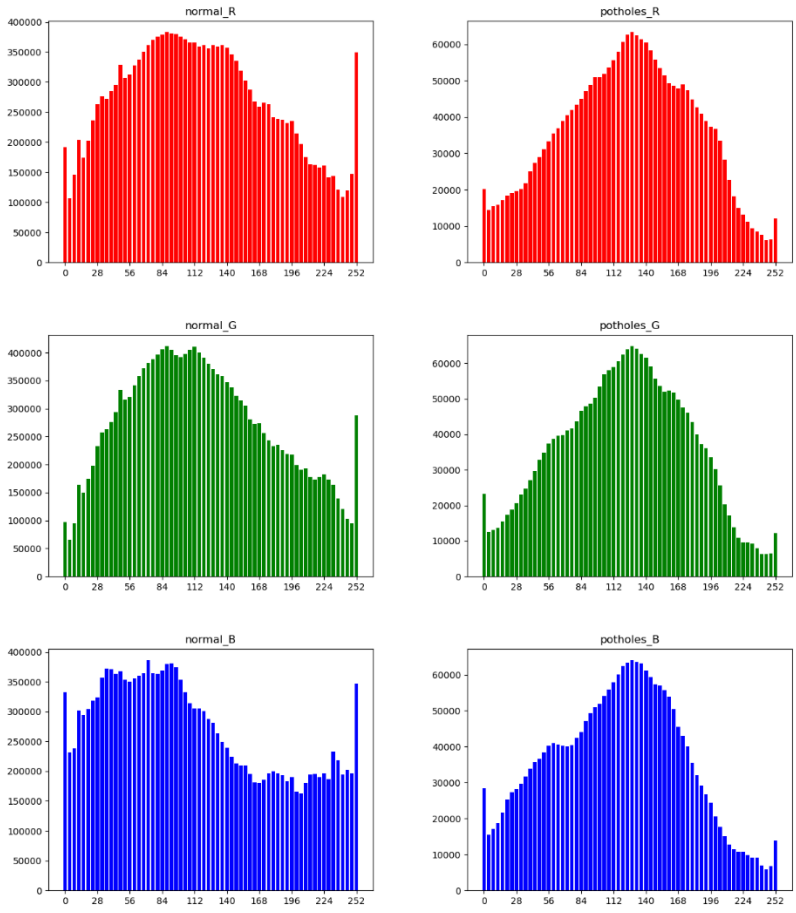


图 5 颜色分布直方图

图 5 中左侧的直方图为 normal 类颜色分布直方图，右侧的直方图为 potholes 类颜色分布直方图。

由此可以得到：

potholes 类直方图的整体分布比 narmal 类直方图的整体分布更靠右，其原因是 potholes 类中坑洼处更多，这些区域由于收到反光的影响，像素值普遍偏高；

通过计算各颜色分布直方图的标准差，得到 normal 类颜色分布直方图的标准差分别为 23391.56、26302.91、20491.49，potholes 类颜色分布直方图的标准差为 4349.04、4627.55、4516.47，对比发现 normal 类的标准差明显比 potholes 类的标准差大，说明 normal 类像素分布更加均匀、对比度小，potholes 类像素分布不均匀、对比度大。

最后将每张图片 RGB 通道像素分布作为特征向量$X_i=(x_{i_1},x_{i_2},\cdots,x_{i_{256}}),i=2,3,4$。

（三）Canny 检测图像轮廓特征

normal 类和 potholes 类最明显的区别为没坑和有坑，直觉上认为道路有坑处会有更明显的轮廓形状，因此使用 Canny 算法对图片进行了轮廓检测，以提取出图像的轮廓特征。其基本流程如下：

1、噪声去除：由于边缘检测对噪声非常敏感，因此首先需要使用高斯滤波器对图像进行平滑处理。高斯滤波基于高斯函数，其形式为$G(x,y)=\frac{1}{2\pi\sigma^2}e^{-\frac{x^2+y^2}{2\sigma^2}}$。其中，$\sigma$是标准差，决定了滤波器的宽度。这个函数的离散近似被用作卷积核。

2、计算梯度强度和方向：使用 Sobel 算子计算图像的梯度。Sobel 算子是两个3×3的矩阵，分别是S_x和S_y。分别用于计算图像x方向像素梯度矩阵G_x和y方向像素梯度矩阵G_y。最后梯度大小通过$G=\sqrt{G_x^2+G_y^2}$计算，方向通过$\theta=\arctan(G_x,G_y)$计算。

3、非极大值抑制：为了得到细致的边缘，需要在梯度方向上进行非极大值一致。在这一步中会检查像素是否是其邻域中沿着梯度方向爱的局部最大值。如果不是，像素被设为 0。

4、设置高阈值和低阈值来确定边缘：基于梯度强度，像素被分类为强、弱或非边缘。强边缘像素的强度高于高阈值，弱边缘像素的强度在两个阈值之间，而非边缘像素的强度低于低阈值。

5、边缘跟踪：弱边缘像素被标记为真实边缘或非边缘像素。基于连通性，如果边缘像素周围有一个强边缘像素，则该像素被标记为真实边缘像素；否则，它被抑制。

调用 Canny 算法对文件“potholes16.jpg”进行识别，结果如下：



图 6 原图（左）和轮廓图（右）

从图 6 中可以看出，坑处的轮廓信息相比于周围的环境更加明显，这与我们的猜测一致。

最后在每张图的轮廓图中寻找封闭的轮廓，计算出所有封闭轮廓总面积与图片面积的比值，并作为一个特征向量$X_5=(x_5)$。得到了 normal 类的平均封闭轮廓面积占比为 0.04899，potholes 类的平均封闭轮廓面积占比为 0.09545，有着显著的区别。

4.1.3 模型建立

为了提高模型的识别率和分类准确度，在建立模型时采用了集成学习中的 bagging 算法来搭建模型。尽管集成学习会对多个个体学习器进行学习，但是由于可以并行学习，所以在训练时间上并不会损失过多。

在个体学习器上选择异质的学习器，即个体学习器的种类并不完全相同。为了避免多个学习器集成后结果反而变得更糟，对于个体学习器需要其具有一定的准确性，同时需要个体学习器间具有差异。因此我们选择了一个 Yolov5 模型、一个 Yolov8 模型、一个基于集成学习的机器学习模型作为个体学习模型。

结合策略上采用加权投票法。虽然可以利用提取的特征使用机器学习算法进行分类，但效果仍然有限，所以在权重上我们将机器学习模型权重设为$\alpha_1=0.2$、Yolov5 模型权重设置为$\alpha_2=0.4$、Yolov8 模型权重设置为$\alpha_3=0.4$。当$\alpha_1X_1+\alpha_2X_2+\alpha_3X_3\geq 0.5$则分为 normal 类、$\alpha_1X_1+\alpha_2X_2+\alpha_3X_3<0.5$则分为 potholes 类。

这样的权重选择就可以解决两个深度学习模型造成分类不同时难以决策的问题：当两个深度学习模型分类结果相同时，结果即为深度学习分类结果；当两个深度学习模型分类结果不同相同时，结果即为机器学习分类结果。

基于集成学习的机器学习模型的结构与上述相同，选择了一个支持向量机 SVM 模型、逻辑回归模型、CART 决策树模型作为个体学习器。结合策略上同样采用了相对多数投票法。

问题一的模型结构如下：

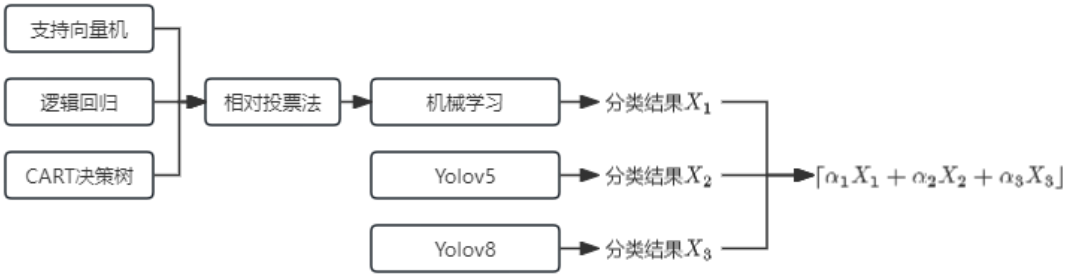


图 7 模型结构图

4.1.4 基于集成学习的机器学习模型

在 Bagging 中，每个个体学习器都是在不同的训练集上独立地进行训练。这些训练集是通过自助采用方法从原始训练集中有放回地抽取得到，因此每个训练集都是从原始训练集中有重复地采样得到的，以增强模型的多样性。最后通过相对投票法得到最终的分类结果。

在训练集上，事先选出的 80%图像作为训练集，每张图片经过特征提取得到的特征向量用于训练个体学习器。剩下 20%图片则作为测试集，用于评估模型的分类效果。

在个体学习器上我们选择了支持向量机 SVM、逻辑回归、CART 决策树这三个个体学习器。

支持向量机是一种用于二分类和多分类问题的有监督学习算法，其目标是通过找到具有最大间隔的决策超平面来实现最大间隔分类。找到最大间隔决策边界可以通过求解对偶问题得到支持向量机的最优解，所以支持向量机算法实际上是通过求解一组二次优化问题来寻找最优的超平面和分类决策函数。其数学原理如下：

$$\begin{aligned} &\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ &\text{s.t.} \sum_{i=1}^{N} \alpha_i y_i = 0 \quad \alpha_i \geqslant 0, i = 1, 2, \cdots, N \end{aligned} \tag{1}$$

求解后即可得到决策超平面：

$$f(x) \quad = w^T \phi(x) + b = \sum_{i=1}^{N} \alpha_i y_i \phi(x_i)^T \phi(x) + b = \sum_{i=1}^{N} \alpha_i y_i k(x, x_i) + b \tag{2}$$

其中核函数$k(x_i, x_j)$采样高斯核函数，即$k(x_i, x_j) = e^{(-\frac{\| x_i - x_j \|^2}{2\sigma^2})}$。

逻辑回归是用于二分类问题的有监督学习算法。其假设函数使用了线性回归的形式，并通过 Sigmod 函数进行映射：

$$h_\theta(x) = sigmoid(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \tag{3}$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} (y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))) \tag{4}$$

使用最大似然估计，通过最小化损失函数$J(\theta)$来估计参数θ。此处选择的优化算法

为梯度下降法。通过最大化似然函数来调整模型参数，使得模型对观测样本的预测概率尽可能接近真实标签的概率。

CART 决策树与一般决策树的区别在于 CART 树是一颗二叉树，在进行分类任务时其选择特征采用基尼系数，基尼系数代表了模型的不纯度，基尼系数越小，不纯度越低，特征越好。

$$Gini(D)=\sum_{i=1}^{n}p(x_i)*(1-p(x_i))=1-\sum_{i=1}^{n}p(x_i)^2 \tag{5}$$

$Gini(D)$ 反映了从数据集 D 中随机抽取两个样本，其类别标记不一致的概率。因此 $Gini(D)$ 越小，则数据集 D 的纯度越高。

4.1.5 基于 Yolov5 的深度学习模型

Yolov5 的结构如图 8 所示。

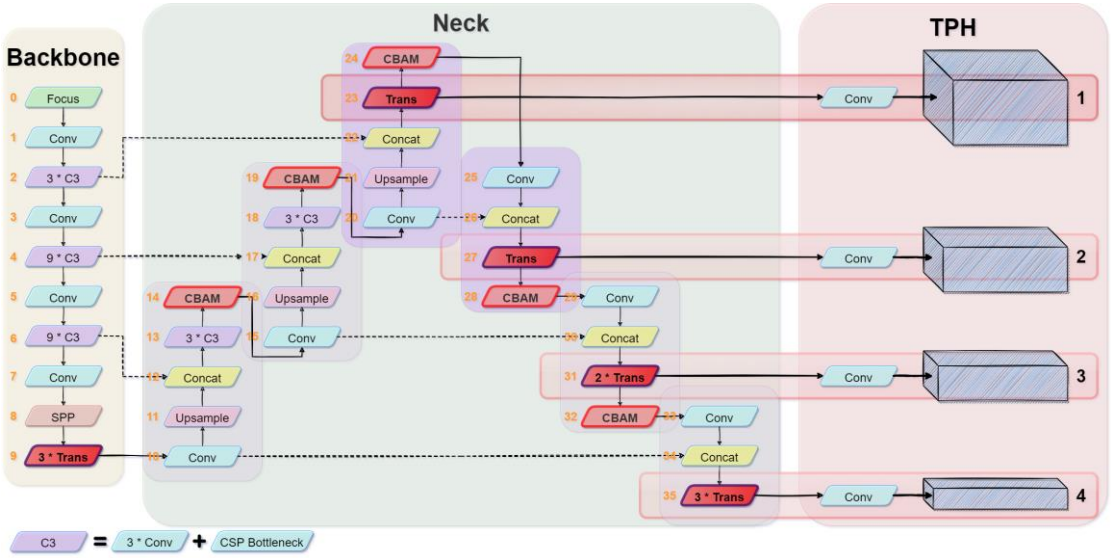


图 8 Yolov5 结构图

(1) 数据输入

Yolov5s 模型在输入端采用了先进的技术，有效地提升了图像数据的信息丰富度，并在目标检测框的调整方面具有更高的灵活性。本文将详细介绍 Yolov5 模型在输入数据上的预处理方法，主要包括 Mosaic 数据增强技巧和自适应锚框计算。

- Mosaic 数据增强

Mosaic 数据增强主要思想是将四张图片进行随机裁剪，再拼接到一张图上作为训练数据。Mosaic 数据增强有助于提高模型的鲁棒性和泛化能力，因为它引入了更多的场景变化和复杂性，避免因部分类型数据过少而导致过拟合，且提高了对小目标物体的检测性能。

在拼接前，程序会生成一个大的合成图像背景，并在背景上随机生成一个点，以该点在图中的横纵坐标作十字分割，对 4 张裁剪后图片所在区域进行划分。在图 10 中，4 张不同样本图的分配位置以不同颜色呈现。其中蓝色点是随机分割点；黑色框是 mosaic 画布；蓝色框是 4 张样本图实际所占空间。由于样本图的裁剪是随机的，难免发生某样

本图大小超过背景的情况，图 11 所示。此时需要对超出部分进行裁剪，将超出部分切除，保留在背景图像上的部分，左上、左下和右下的部分同理。



图 9 Mosaic 样例

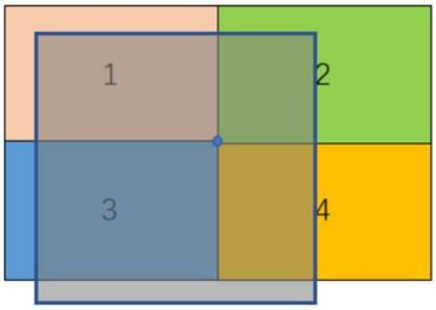


图 10 Mosaic 分割思路

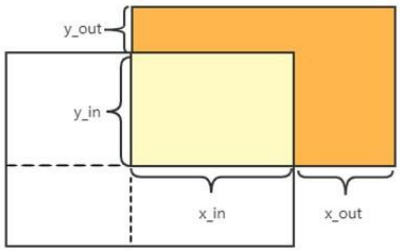


图 11 Mosaic 分割实例

- 自适应锚框计算

大部分传统的图像检测模型往往只能对固定尺寸图像进行单目标检测。而在数据集中，训练集样本的每一目标框的尺寸、数量、位置不一，难以具体确定锚框大小，目标框的尺寸没有规律，加上图片尺寸较大，不能通过简单的数值约定约束锚框大小。因此，自适应锚框计算的核心思想是根据数据集中的目标尺寸和分布情况来自动计算和调整锚框的大小和长宽比。

YOLOv5s 网络在每次训练迭代中，按照事先定义的输入长宽尺寸设置锚框，然后根据新数据集的标签信息，自动学习适应锚框的尺寸。如果召回率达到了 0.98 以上，就不再进行优化，而是直接返回结果；但如果召回率低于 0.98，那么模型将采用 K 均值聚类方法，结合自定义数据集的分析，以获得适合数据集目标边界框的预设锚定边框。具体来说，在可能的位置根据预设锚定框来选取目标区域，然后在这些预设锚定框的基础上进行微调。其次，利用真实标签位置相对于预设锚定框的偏移来构建预定义的锚框，模型在这些锚框的基础上输出目标检测的预测框。接下来，将这些预测框与真实标签进

行比对，计算它们之间的差异，并通过反向传播来更新模型参数，迭代使损失值最小化，以使预测框尽可能与初始的输入锚框相匹配。

这一过程实现了自适应计算不同训练数据集中的最佳锚框值。在目标检测任务中，首先生成多个锚框，然后预测每个锚框的类别信息和位置偏移量。根据这些预测的偏移量，调整锚框的位置，从而得到最终的预测边界框。最后，通过筛选，确定需要输出的预测边界框。

(2) Focus 结构分析

Focus 结构是 Yolov5s 的创新点，在历史 Yolo 系列的模型中未出现此种结构。其切片操作如图所示。具体操作是在一张图片中每跨步一个像素取出一个值，类似于邻近下采样，得到四张互补的图片，并未产生信息丢失。这样一来，将 W、H 信息就集中到了通道空间，输入通道扩充了 4 倍，即拼接起来的图片相对于原先的 RGB 三通道模式变成了 12 个通道，最后将得到的新图片进行卷积操作，最终得到了没有信息丢失情况下的二倍下采样特征图（Focus 下采样的参数量略多于 Conv 下采样，但是有一定提升效果）。

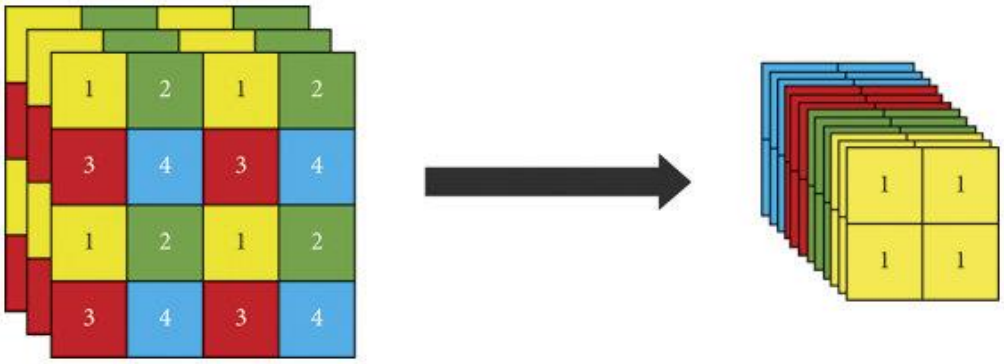


图 12 Focus 切片操作

在模型中，原始 608*608*3 的图像输入 Focus 结构，采用切片操作，先变成 304*304*12 的特征图，再经过一次 80 个卷积核的卷积操作，最终变成 352*352*80 的特征图。其流程如图 13 所示

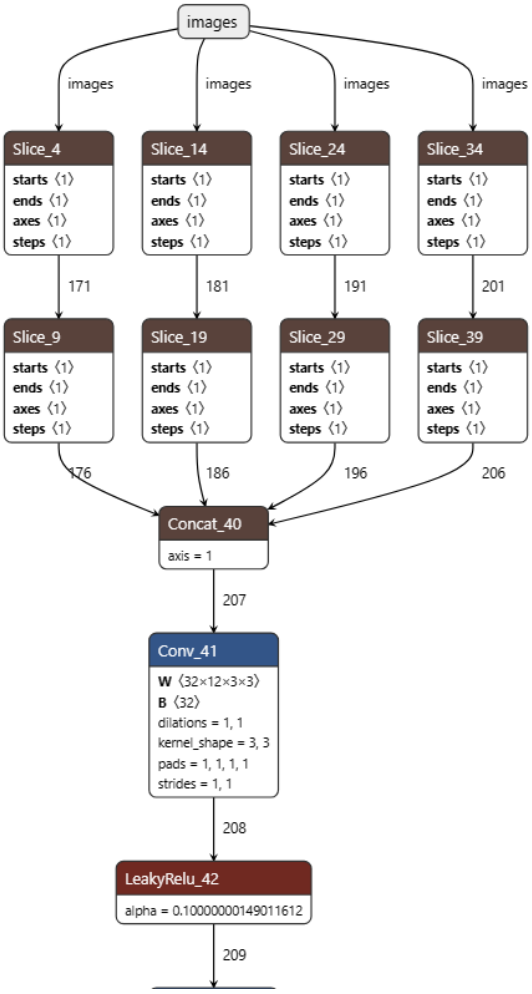


图 13 Focus 结构示意图

(3) Backbone 骨干网络

Backbone 骨干网络是 Yolov5s 中一个预先训练的网络，其主要负责提取图像数据的丰富特征表示，得到多个大小不同的特征图，这有助于降低图像的空间分辨率并提高其特征（通道）分辨率。增加残差结构，可以增加层与层之间反向传播的梯度值，避免因为加深而带来的梯度消失，从而可以提取到更细粒度的特征且不必担心网络退化。Backbone 的网络结构其大致流程如图 14 所示（输入图片尺寸为 608x608）：

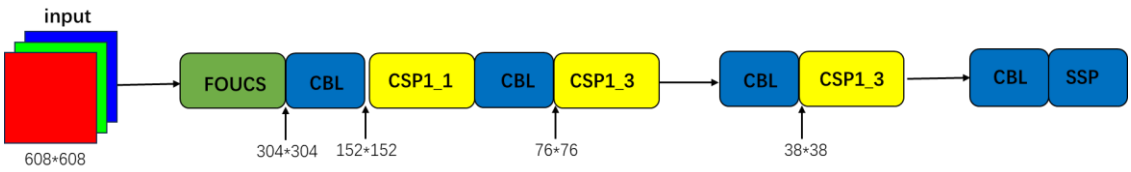


图 14 Backbone 骨干网络结构示意图

(4) Neck 网络

Yolov5s 的 Neck 最新和 Yolov4 中一样，都采用 FPN+PAN 的结构。Yolov4 在 FPN 层的后面还添加了一个自底向上的特征金字塔。FPN 的功能在于有效实现了从高维度向低维度的语义信息传递，加强了不同层次特征的融合，从而实现了多尺度的目标预测，

从而使模型在大目标的识别方面表现更为明确；而 PAN 则在低维度向高维度再次传递语义信息，从而增强了模型对小目标的识别准确性。这两种结构的联合应用使得不同特性互相补充，从而进一步提升了模型的性能。

由图 15 所示，FPN 是自顶向下的，将高层的特征信息通过上采样的方式进行传递融合，得到预测的特征图。每个 CSP 模块前面的卷积核大小都是 3*3，步长为 2，相当于下采样操作。因此得到的特征图的大小依次为 76*76、38*38、19*19。

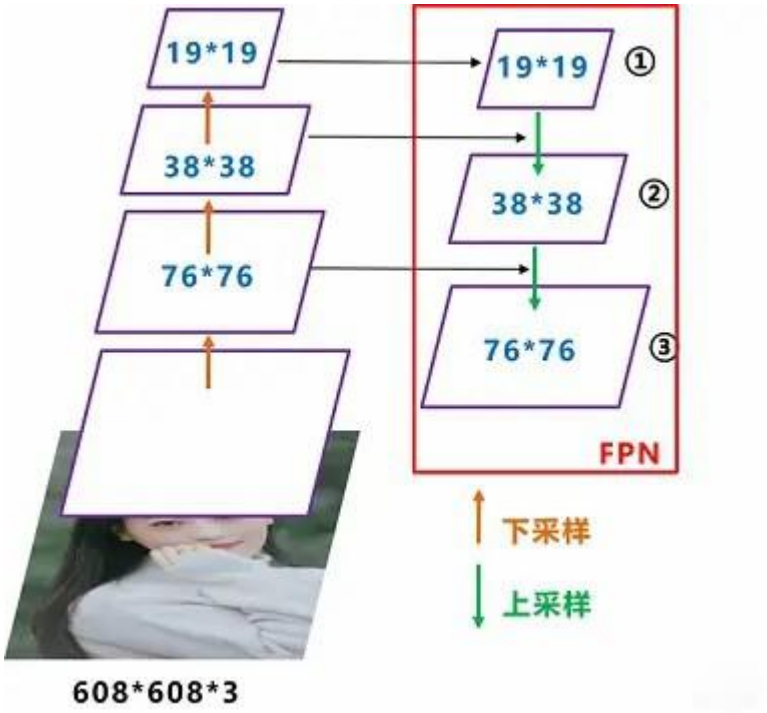


图 15 FPN+PAN 结构示意图

和 Yolov3 的 FPN 层不同，Yolov5s 在 FPN 层的后面还添加了一个自底向上的特征金字塔。其中包含两个 PAN 结构。这样结合操作，FPN 层自顶向下传达强语义特征，而特征金字塔则自底向上传达强定位特征，二者协同，从不同的主干层对不同的检测层进行参数聚合。

Yolov5 的 Neck 结构中，采用借鉴了 CSPnet 设计的 CSP2_X 结构。与 CSP1_X 相比，CSP2_X 将 Bottleneck 部分换成了 2x 次 CBL 操作。具体见图 16。

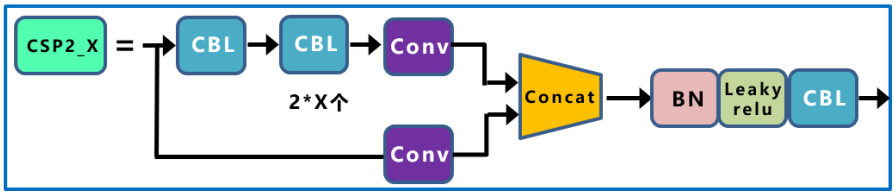


图 16 CSP2_X 结构示意图

4.1.6 基于 Yolov8 的深度学习模型

Yolov8 是一种基于深度学习的目标检测算法，它是 Yolo 系列算法的最新版本。Yolov8 引入了新的功能和改进，进一步提升性能和灵活性，使其成为实现目标检测、图像分割、姿态估计等任务的最佳选择。其结构图如图 17 所示[1]。

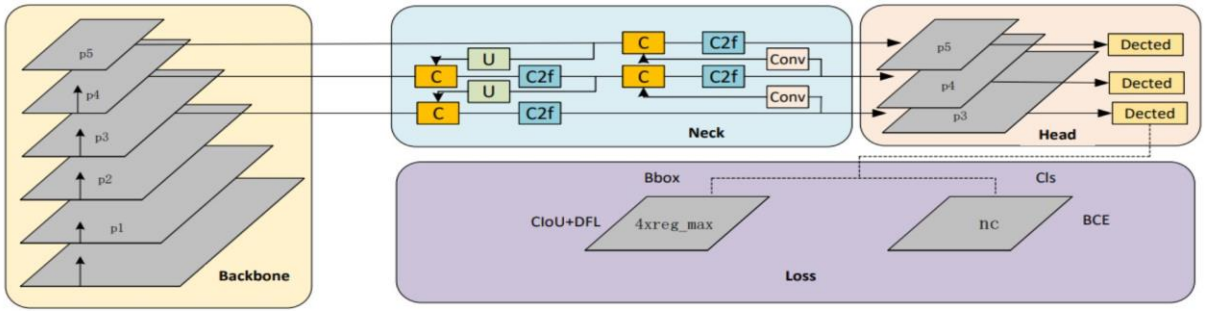


图 17 Yolov8 网络结构图

相较于 Yolov5 模型，Yolov8 模型的改进主要在于 Backbone、Neck、Head 和 Loss 等方面。

(1) Backbone

Yolov8 的 Backbone 部分与 Yolov5 基本相同，基于 CSP 思想，将 C3 模块替换为 C2f 模块。C2f 模块借鉴了 Yolov7 中的 ELAN 思路，将 C3 和 ELAN 结合在一起组成了 C2f 模块，使 Yolov8 在保证自身重量轻的同时可以获得更丰富的梯度流信息。

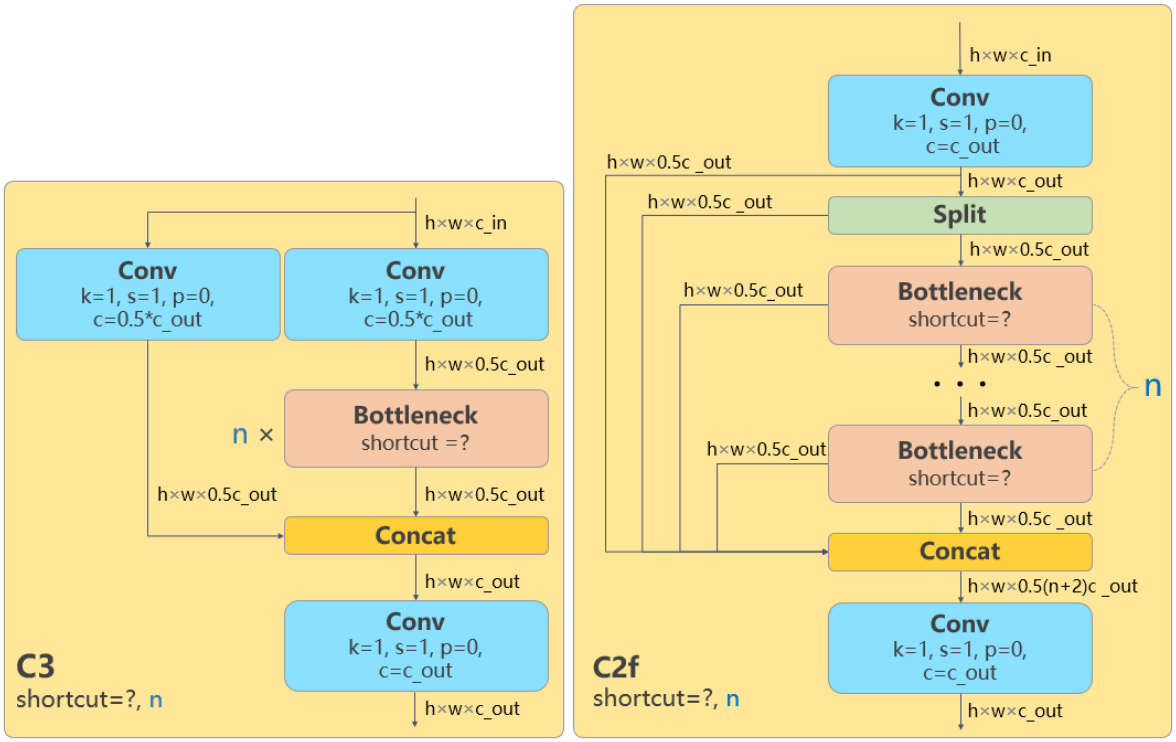


图 18 C3 与 C2f 对比图

在 Backbone 末端仍然使用最流行的 SPPF 模块，依次传递 3 个大小为 5*5 的 Maxpools，然后将每一层进行串联，这样既保证了不同尺度下物体的精度，同时有保证了物体的轻量化。

(2)Neck

在 Neck 部分，Yolov8 使用的特征融合方法仍然是 PAN-FPN，加强了不同尺度下特征层信息的融合和利用。Yolov8 使用了两个上采样和多个 C2f 模块以及最终解耦的头部结构来组成 Neck 模块。

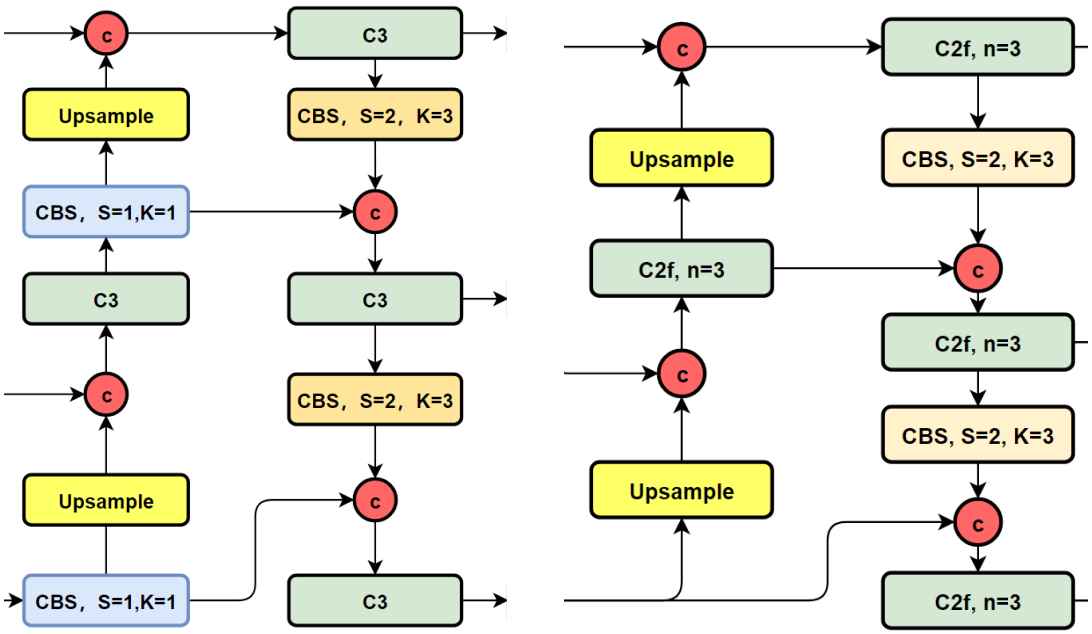


图 19 Yolov5(左)和 Yolov8(右)Neck 结构对比图

(3)Head

Yolov8 与其前身 Yolov5 相比，最重要的改进在于 Head 部分。具体而言，通过引入一种解耦头结构，类似于 Yolox，Yolov8 成功将回归分支和预测分支进行了有效分离。此外，针对回归分支的处理中，Yolov8 采用了 Distribution Focal Loss（DFL）策略中提出的积分形式表示法，将原本以确定性单值作为预测标准的回归坐标转化为一个分布，从而进一步提高了模型的精度和鲁棒性。

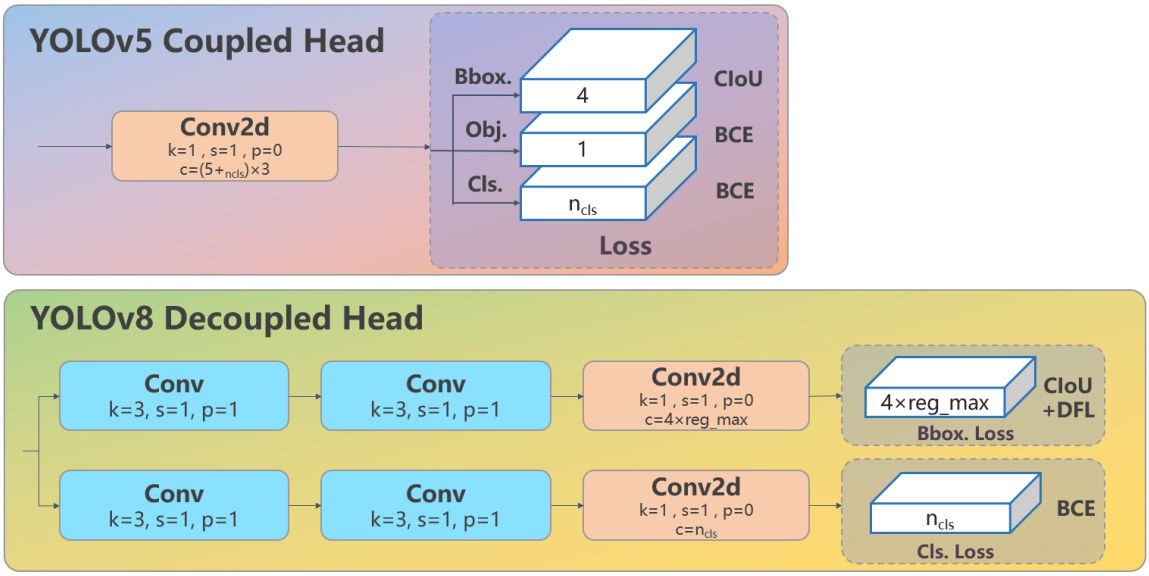


图 20 Yolov5 与 Yolov8 的 Head 结构对比图

4.2 问题二的模型建立与求解

4.2.1 模型训练

我们将经过翻转和裁剪操作扩充的数据集按 8:2 的比例划分训练集和测试集，并将问题一中构建的基于集成学习的机器学习、Yolov5 和 Yolov8 三种模型进行训练。

4.2.2 模型的指标评价

(1) 基于集成学习的机器学习模型

表 1 机器学习模型的评价指标表

准确率	精确率	召回率	F1 值
0.6179	0.6646	0.7004	0.6821

准确率（Accuracy）为 0.6179，表示模型在整体样本中预测正确的比例较低。

精确率（Precision）为 0.6646，表示在所有预测为正类的样本中，有 66.46%是真正的正类。较高的精确率意味着模型在预测为正类的样本中的误判较少。

召回率（Recall）为 0.7004，表示在所有真实正类样本中，有 70.04%被正确预测为正类。较高的召回率意味着模型能够较好地捕捉到正类样本。

F1 分数（F1 Score）为 0.6821，综合考虑了精确率和召回率。F1 分数是精确率和召回率的调和平均值，用于综合评估模型的性能。较高的 F1 分数表示模型在同时考虑精确率和召回率时取得了较好的平衡。

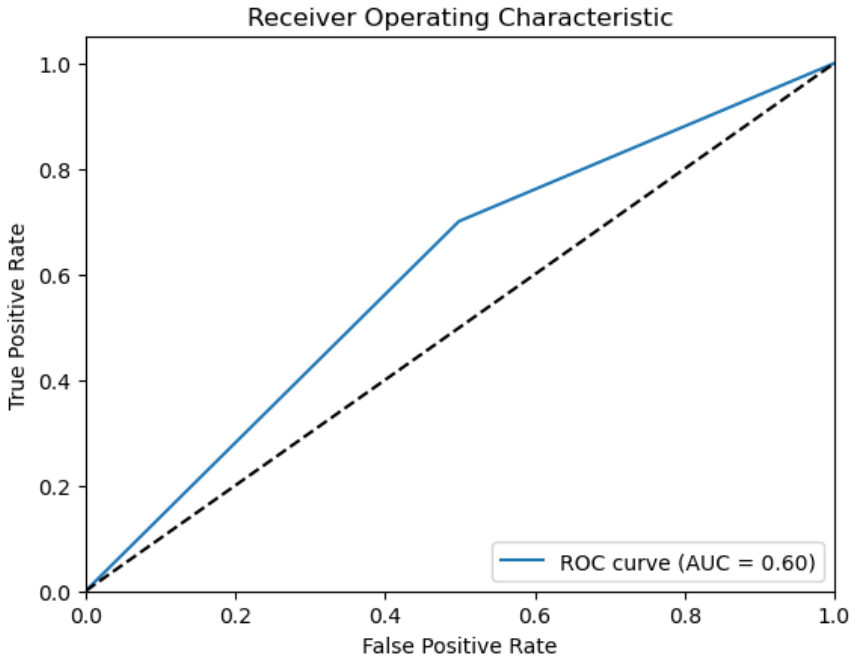


图 21 ROC 曲线图

该 ROC 曲线大部分时间都位于对角线之上，这表示模型的预测性能超过了随机预测。AUC (Area Under Curve) 值是 0.60。AUC 的范围是 0 到 1。一个完美的模型的 AUC 是 1，而一个完全随机的模型的 AUC 是 0.5。AUC 值为 0.60 表示该模型的性能比随机预测要好，但仍有提升的空间。

(2) 基于 Yolov5 的深度学习模型

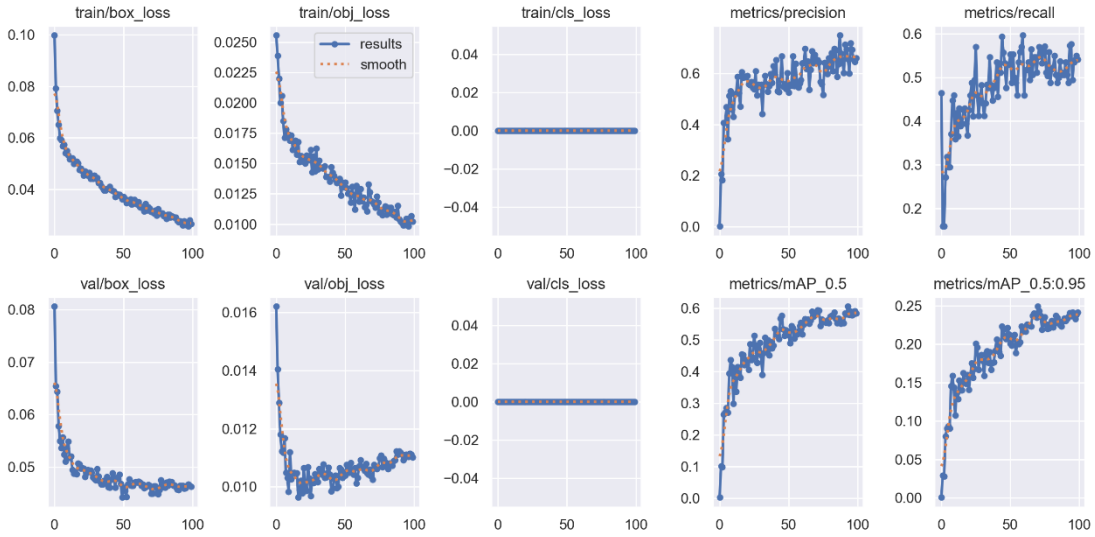


图 22 Yolov5 性能模型曲线

train/box_loss 和 val/box_loss：这两张图展示了模型的 bounding box 预测损失。从图中可以看出，训练损失随着训练周期数的增加呈下降趋势，验证损失也有相似的趋势。

train/obj_loss 和 val/obj_loss：这些图展示了模型预测物体存在的损失。同样，训练和验证的损失都在下降，但是在验证过程中下降得更为显著。

train/cls_loss 和 val/cls_loss：这是模型的分类损失。从图中可以看出，分类损失在训练和验证过程中基本保持不变。可能的原因是类别非常均衡，或者模型在分类任务上已经达到了很好的效果。

metrics/precision 和 metrics/recall：这两张图表示模型的精确度和召回率。精确度和召回率是评价模型性能的关键指标，特别是在物体检测任务中。从图中可以看出，随着训练的进行，精确度和召回率都有所波动，但总体上呈上升趋势。

metrics/mAP_0.5 和 metrics/mAP_0.5:0.95：这两张图展示了模型的 mean Average Precision (mAP)。mAP 是评价物体检测模型性能的一个重要指标。第一张图（mAP_0.5）只考虑 IoU>0.5 的预测，而第二张图（mAP_0.5:0.95）则考虑了从 0.5 到 0.95 的所有 IoU 值。后者提供了一个更全面的评估。两者都显示了随着训练周期的增加，mAP 也在逐渐上升，这意味着模型的预测效果在改善。

这组图表显示，随着训练周期数的增加，Yolov5 模型的性能在不断提高。所有的损失都在下降，而精确度、召回率和 mAP 等性能指标都在上升。

(3) 基于 Yolov8 的深度学习模型

Yolov8 模型的 F1-Confidence Curve、Precision-Confidence Curve、Precision-Recall Curve 和 Recall-Confidence Curve 性能曲线图如下：

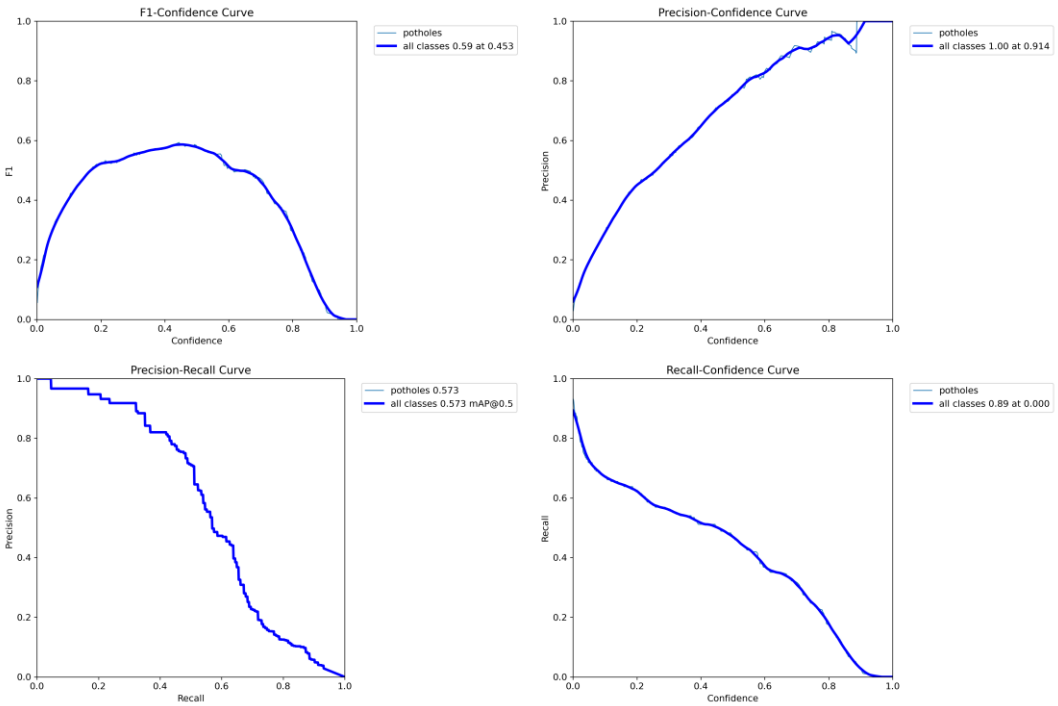


图 23 Yolov8 模型性能曲线 1

F1-Confidence Curve：对于“potholes”类别，当置信度在约 0.453 时，F1 分数达到了其最大值，约为 0.59。这意味着模型对“potholes”分类的综合性能（精确度和召回率的结合）在这个置信度下最好。F1 曲线的形状呈现了一个明显的山峰形状，随着置信度的增加，F1 分数先增加然后下降。

Precision-Confidence Curve：对于“potholes”类别，随着置信度的增加，精确度呈现出一个上升的趋势，并在约 0.914 的置信度处达到 1.0，这意味着在这个置信度水平上，模型的精确度达到了最大值。

Precision-Recall Curve：这条曲线展示了精确度和召回率之间的权衡关系。对于“potholes”类别，其最大的平均精确度（mAP）为 0.573，这是一个中等的性能指标。曲线从左上角向右下角移动，表示在提高召回率的过程中，精确度有所下降。

Recall-Confidence Curve：当置信度为 0 时，对于“potholes”类别的召回率为 0.89。随着置信度的增加，召回率逐渐下降。这意味着在高置信度下，模型可能错过了一些真正的“potholes”实例。

Yolov8 模型在“potholes”分类上的表现呈现出一定的权衡。虽然在某些置信度水平上，它的精确度可以达到很高，但其召回率可能较低。

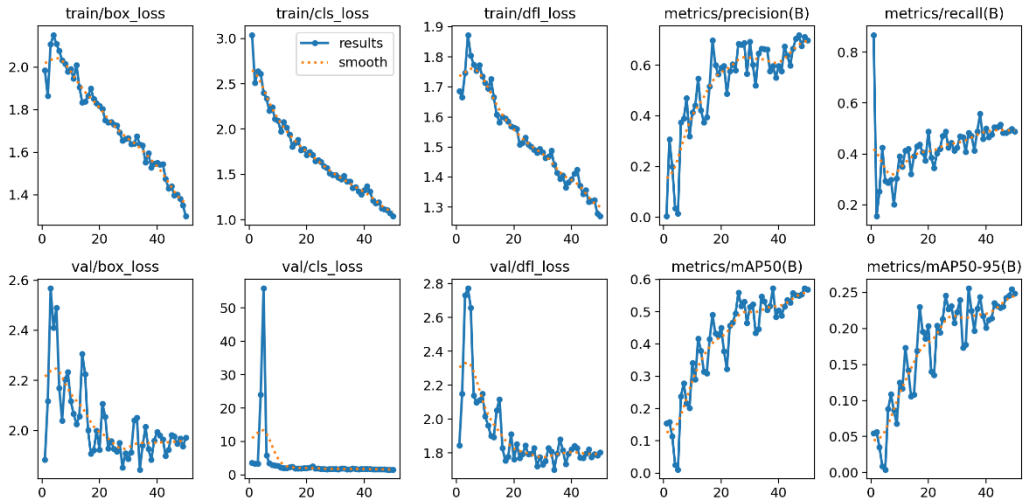


图 24 Yolov8 模型性能曲线 2

train/box_loss：训练过程中的框损失 (box loss) 显示了一个下降的趋势，这表示模型在学习过程中在预测边界框位置上逐渐变得更准确。

train/cls_loss：分类损失 (class loss) 也呈现下降趋势，这意味着模型在区分不同的类别上变得更好。

train/dfl_loss：这个损失也呈现了下降趋势。

验证损失 (Validation Loss)：val/box_loss, val/cls_loss,和 val/dfl_loss：验证集上的损失在初始阶段有较大的波动，但随着时间的推移，这些波动逐渐减小，而且呈下降趋势。这意味着模型在验证集上的性能逐渐稳定并且提高。

metrics/precision(B)和 metrics/recall(B)：在训练过程中，精确度 (precision) 和召回率 (recall) 呈现了波动的上升趋势。这表示模型在正确识别目标的能力逐渐增强。

metrics/mAP50(B)：这个指标代表在 IoU 阈值为 0.5 时的平均精确度 (mean average precision)。它也显示了一个上升的趋势，说明模型的整体性能在提高。

metrics/mAP50-95(B)：这是在多个 IoU 阈值（从 0.5 到 0.95）上的平均精确度。这个指标也在逐渐上升，表示模型在多个阈值上的性能都在提高。

Yolov8 模型在训练过程中逐渐获得了更好的性能。损失值在下降，而性能指标在上升。但是，还需要注意验证损失和性能指标的波动，这可能表示模型对于训练数据可能存在一定的过拟合。

4.2.3 模型的鲁棒性

鲁棒性是指系统或模型在面对各种不确定性、异常情况或攻击时的稳定性和健壮性。在深度学习和人工智能领域，鲁棒性是指模型对于输入数据的变化、扰动、噪声以及对抗性攻击等的处理能力。验证模型的鲁棒性是一个重要的任务，本文通过添加高斯噪声和生成对抗样本两种方法来验证模型的鲁棒性。

4.2.4 添加高斯噪声

高斯噪声（Gaussian noise）是一种常见的随机噪声模型，也被称为正态分布噪声。它是指服从高斯分布的随机变量所引入的噪声。



图 25 高斯噪声

我们使用训练后的模型分别对提供的 301 个原始图像文件进行分类预测并计算其准确率，接着对该数据集添加高斯噪声重新进行分类预测并计算准确率，对比准确率的变化而评价模型的鲁棒性。

- 基于 Yolov5 的深度学习模型

原始图像分类准确率：97.010%，添加噪声图像分类准确率：97.010%。

表 2 Yolov5 模型有无噪声识别对比表（识别结果不同部分）

图像文件名称	无噪声	有噪声
Normal9.jpg	1	0
normal227.jpg	0	1
normal268.jpg	0	1
normal309.jpg	1	0

Yolov5 模型在面对噪声时表现出了一定的鲁棒性。尽管无噪声和有噪声的识别准确率相同，但仍 4 个样本在两种情况下的识别结果不一致。这意味着模型在面对噪声时可能会受到一些干扰，导致分类结果与无噪声情况下有所不同。

- 基于 Yolov8 的深度学习模型

原始图像分类准确率：99.003%，添加噪声图像分类准确率：96.678%。

表 3 Yolov8 模型有无噪声识别对比表（识别结果不同部分）

图像文件名称	无噪声	有噪声
normal106.jpg	1	0
normal14.jpg	1	0
normal151.jpg	1	0
normal177.jpg	1	0
normal190.jpg	1	0
normal208.jpg	1	0
normal297.jpg	1	0
potholes00000.jpg	1	0
potholes00024.jpg	0	1

通过比较原始图像分类准确率和添加噪声图像分类准确率，我们可以评估模型的鲁棒性。添加噪声导致图像分类准确率下降了约 2.325%。在这种情况下，96.678% 的分类准确率表明 Yolov8 模型在存在噪声的图像上仍然能够较好地进行分类。虽然添加噪声导致准确率下降，但仍然保持了相对较高的分类性能。

4.2.5 生成对抗样本

生成对抗样本是深度学习邻域的热门研究话题。对抗样本是通过在原始输入数据上添加微小的扰动来生成的，这些扰动足以欺骗模型，使其产生不正确的预测，但对人类

来说这种变化是几乎不可察觉的。深度神经网路在许多任务上表现出色，但对抗样本揭示了它们的脆弱性，对抗攻击可能会影响现实世界的应用，例如自动驾驶、面部识别。其中坑洼道路就被广泛应用于自动驾驶领域，考虑到现实世界中安全性的要求，此处我们对模型进行了对抗样本攻击，以评估模型的安全性。

现有的对抗样本大多是通过在正常图像上叠加扰动生成的，在保证人类无法察觉的同时，使模型尽可能识别错误。在生成对抗样本时实际上是解决一个优化问题，因此我们需要对一个目标函数最小化以达到目标效果。通用的优化目标函数$f(\cdot)$定义公式如下：

$$f(x,\delta,t)=l_{model}(F(x+\delta),t)+c\cdot l_{metric}(x+\delta) \tag{6}$$

其中$F(x+\delta)$是图像识别模型；c是权衡攻击成功和最终图像的权重系数；l_{model}是与识别模型相关的损失函数，用来衡量期望的模型输出t和实际输出$F(x+\delta)$之间的差异；$l_{mrtric}(x+\delta)$是模型对样本$x+\delta$的置信度。

基于遗传算法的生成对抗样本的优点是它不需要直接访问模型的参数或内部结构。这使其特别适合于黑盒攻击情境。此外，遗传算法提供了一种灵活的方法来调整和探索不同的对抗扰动，这有助于找到有效的对抗样本。算法流程图如下：

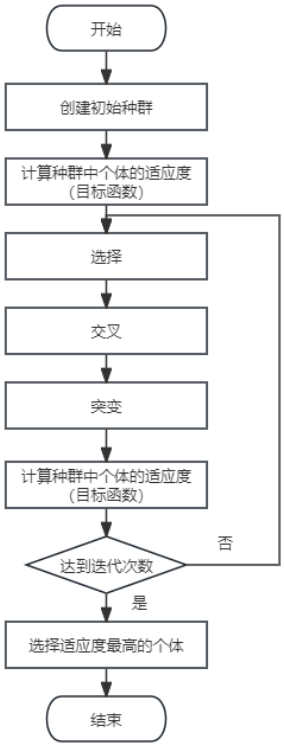


图 26 遗传算法流程图

我们对 Yolov5、Yolov8 这两个深度学习模型分别进行了生成对抗样本攻击，将初始种群数量设置为 100、迭代次数为设置 500、扰动系数设置为 1，在原始 pothels 类中 35 张图片生成了对抗样本。结果如下：

表 4 生成对抗样本结果

文件名	v5	v5 对抗样本	v8	v8 对抗样本
potholes00000.jpg	1	1	1	0
potholes00001.jpg	1	1	1	1
potholes00002.jpg	1	1	1	1
potholes00003.jpg	1	1	4	1
potholes00004.jpg	0	0	2	2
potholes00007.jpg	0	0	0	0
potholes00008.jpg	1	1	1	1
potholes00009.jpg	2	2	2	2
potholes00010.jpg	2	2	1	1
potholes00012.jpg	1	1	1	1
potholes00013.jpg	1	1	1	1
potholes00018.jpg	1	1	1	1
potholes00019.jpg	1	1	1	1
potholes00020.jpg	1	1	1	1
potholes00024.jpg	2	2	1	1
potholes00028.jpg	2	2	2	2
potholes1.jpg	2	2	2	2
potholes12.jpg	4	4	2	2
potholes16.jpg	1	1	1	1
potholes19.jpg	6	6	2	2
potholes2.jpg	8	8	2	2
potholes22.jpg	3	3	4	4
potholes24.jpg	9	9	10	10
potholes26.jpg	1	1	2	2
potholes29.jpg	5	5	8	8
potholes3.jpg	27	7	18	18
potholes31.jpg	3	3	2	2
potholes32.jpg	7	7	9	9
potholes34.jpg	1	1	1	1
potholes4.jpg	5	5	3	3
potholes5.jpg	1	1	1	1
potholes6.jpg	2	2	2	2
potholes7.jpg	1	1	1	1
potholes8.jpg	9	9	7	7
potholes9.jpg	1	1	1	1

表中的数字表示模型识别出的坑数。从结果上看，Yolov5 模型在受到对抗样本攻击后有 1 个文件识别结果出现了差异，为文件“potholes3.jpg”；Yolov8 模型在受到对抗样本攻击后有 2 个文件识别后出现了差异，为文件“potholes00000.jpg”、“potholes00003.jpg”。

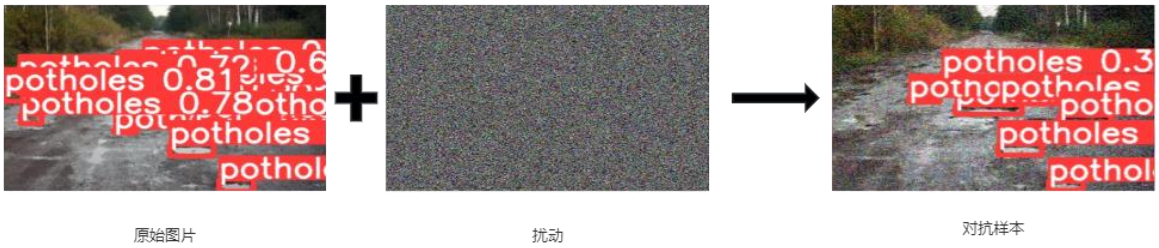


图 27 Yolov5 生成对抗样本结果

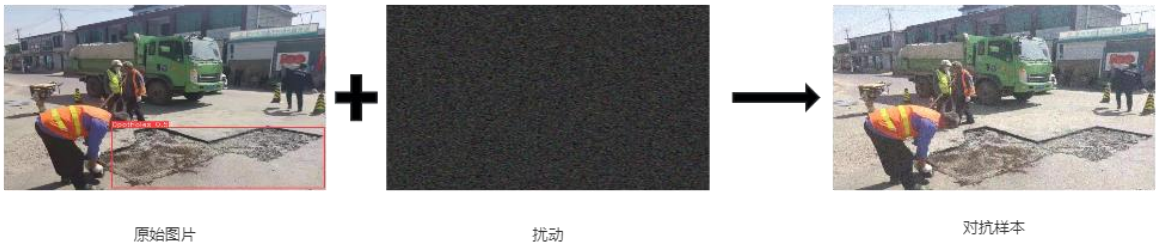


图 28 Yolov8 生成对抗样本结果

文件“potholes3.jpg”在生成对抗样本后，Yolov5 的识别结果从 27 个坑洼变为了 7 个坑洼；文件“potholes00000.jpg”在生成对抗样本后，Yolov8 的识别结果从 1 个坑洼变为了 0 个坑洼。

不考虑对抗攻击强度因素，从总体结果来看，生成的对抗样本对原始识别结果影响较小，该模型具有一定的抗对抗样本的能力，鲁棒性较强。

4.3 问题三的模型建立与求解

本题我们利用上述构建的机器学习、Yolov5 和 Yolov8 三个模型对测试数据集进行分类，其中机器学习模型权重为 0.2，Yolov5 和 Yolov8 模型的权重均为 0.4。若这三个模型的加权投票大于或等于 0.5，则分为正常道路，反之则为坑洼道路。在该模型下，若 Yolov5 模型和 Yolov8 模型分类结果一致，则选用该分类结果，否则选用机器学习的分类结果。部分分类结果如表 5，全部分类结果见“test_result.csv”文件。

表 5 模型分类结果

图像名称	机器学习	Yolov5	Yolov8	加权投票	分类
00aqq56m.jpg	1	1	1	1	1
00c9sdfq.jpg	1	0	0	0.2	0
00jjy9i3.jpg	1	1	1	1	1
00q3q8py.jpg	1	0	1	0.6	1
00yf8rxj.jpg	1	0	0	0.2	0
01jyy8yk.jpg	1	1	1	1	1
01lx7dej.jpg	0	0	1	0.4	0
01paoyy0.jpg	0	1	0	0.4	0
02f4ee5p.jpg	1	1	1	1	1
02ld1b80.jpg	1	1	1	1	1
		……			
zx9exkdw.jpg	1	0	0	0.2	0

zxfhwbki.jpg	1	0	0	0.2	0
zxgm3ar3.jpg	1	1	1	1	1
zy7m9c8x.jpg	0	1	0	0.4	0
zynixpux.jpg	1	0	0	0.2	0
zyylqfs0.jpg	0	0	0	0	0
zzhe6767.jpg	0	0	0	0	0
zzipq9zn.jpg	0	1	1	0.8	1
zzvom0ax.jpg	0	0	0	0	0
zzvth22q.jpg	1	1	1	1	1

五、 模型评价与推广

5.1 模型优点

1. 本文主要实现了基于 YOLO 目标检测深度学习模型的道路坑洼检测项目，并结合机器学习进行集成学习，所成的融合模型识别率高、分类准确，能够快速准确地识别图像中的指定物体。

2. 在前期数据预处理中，由于所给训练数据集中正常和坑洼道路类别不平衡。我们通过翻转、裁剪操作对 potholes 类数据集进行扩增，显著提高模型预测的准确性。

3. 我们通过对图片增加噪声和生成对抗样本来评估模型的鲁棒性。通过数据对比可得其在处理复杂背景、部分遮挡和小目标等情况下表现出较强的鲁棒性和抗干扰能力，能够适应不同的场景。

5.2 模型缺点

1. YOLOv5 的性能高度依赖于大规模、多样化的训练数据。如果训练数据不足，模型的性能可能会下降。

2. YOLOv5 深度学习模型在处理大规模数据时需要较强的计算资源（GPU）支持，这限制了模型训练的最终性能，使得模型特征提取能力和识别准确度受到进一步限制。

3. 由最终得取的测试结果说明模型的泛化预测能力还需进一步提升。

5.3 模型推广

深度学习在大数据中具有较强的特征提取能力，极大地提高了对于复杂数据集的处理能力，以较快的速度和准确率平衡而广受推崇。由于 YOLO 模型泛化能力、特征提取能力较强，其对于大多数目标检测准确率都较高，这使其应用广泛，其可应用于各种物体检测项目，比如道路人流检测，人脸识别技术等领域。

六、 参考文献

[1]倪云峰,霍洁,侯颖等.基于 YOLOv8—OCR 的井下人员检测算法[J/OL].无线电工程:1-8[2023-11-02].

[2]闫佳,聂楚江,苏璞睿.基于遗传算法的恶意代码对抗样本生成方法[J].电子与信息学报,2020,42(9):2126-2133.

[3]孟东宇. 黑盒威胁模型下深度学习对抗样本的生成[J]. 电子设计工程, 2018, 26(24): 164-167.

[4]任安虎,姜子渊,马晨浩.基于改进 YOLOv5s 的道路裂缝检测算法[J/OL].激光杂志:1-7[2023-11-03].

[5]刘彦清. 基于 YOLO 系列的目标检测改进算法[D].吉林大学,2022

[6]谈世磊,别雄波,卢功林等.基于 YOLOv5 网络模型的人员口罩佩戴实时检测[J].激光杂志,2021,42(02)

[7]Zhu, X., Lyu, S., Wang, X., & Zhao, Q. (2021). "TPH-YOLOv5: Improved YOLOv5 Based on Transformer Prediction Head for Object Detection on Drone-Captured Scenarios." In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops (pp. 2778-2788).

附录

附录 1：canny_形状特征.py

使用 canny 算法提取轮廓，并计算封闭轮廓面积占比

```
import cv2
import os
import pandas as pd

# 图像处理函数
def count_shapes_above_threshold(image_path, threshold):
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # 检查图像是否正确加载
    if image is None:
        print(f"Failed to load image: {image_path}")
        return 0

    # 使用 Canny 算法进行边缘检测
    edges = cv2.Canny(image, 100, 150)

    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    image_area = image.shape[0] * image.shape[1]
    count = 0
    total_area = 0
    for contour in contours:
        area = cv2.contourArea(contour)
        total_area += area
        # if area / image_area > threshold:
        #     count += 1
    return total_area / image_area
# 遍历目录并统计每张图片中的形状数量
def count_shapes_for_each_image(directory, threshold=0.05):
    results = {}
    for root, dirs, files in os.walk(directory):
        for file in files:
            if file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_path = os.path.join(root, file)
                count = count_shapes_above_threshold(image_path, threshold)
```

```
                results[image_path] = count
    return results


# 阈值设定
area_ratio_threshold = 0.01

# 统计
data_path = "testdata_V2"
normal_results = count_shapes_for_each_image(os.path.join(data_path, "normal"), area_ratio_threshold)
potholes_results = count_shapes_for_each_image(os.path.join(data_path, "potholes"), area_ratio_threshold)

# 结果整合并保存为 Excel 文件
df_normal = pd.DataFrame({
    "Image Path": list(normal_results.keys()),
    "Count": list(normal_results.values())
})
df_potholes = pd.DataFrame({
    "Image Path": list(potholes_results.keys()),
    "Count": list(potholes_results.values())
})

# 使用 Excel 的不同工作表保存结果
with pd.ExcelWriter("results.xlsx") as writer:
    df_normal.to_excel(writer, sheet_name="Normal", index=False)
    df_potholes.to_excel(writer, sheet_name="Potholes", index=False)
```

附录 2：翻转.py

将所给的 potholes 图片翻转

```
from PIL import Image
import os

# 定义翻转函数
def flip_image(image_path):
    image = Image.open(image_path)
    # 水平翻转
    flipped_image = image.transpose(Image.FLIP_LEFT_RIGHT)
    # 生成新的文件名，加上_flip 后缀
    new_filename = os.path.splitext(image_path)[0] + '_flip' +
```

```
os.path.splitext(image_path)[1]
    # 保存翻转后的图片
    flipped_image.save(new_filename)

#文件夹路径
folder_path = 'potholes'

# 遍历文件夹中的所有图片文件
for filename in os.listdir(folder_path):
    if filename.endswith('.jpg') or filename.endswith('.png'):
        # 获取图片文件的完整路径
        image_path = os.path.join(folder_path, filename)
        # 对图片进行翻转，并保存到新文件中
        flip_image(image_path)
```

附录 3：裁剪.py

将所给的 potholes 图片四等份裁剪

```
from PIL import Image


def split_image(image_path):
    # 打开原始图片
    image = Image.open(image_path)

    # 获取图片尺寸
    width, height = image.size

    # 计算每个子图的尺寸
    sub_width = width // 2
    sub_height = height // 2

    # 分割图片
    upper_left = image.crop((0, 0, sub_width, sub_height))
    upper_right = image.crop((sub_width, 0, width, sub_height))
    lower_left = image.crop((0, sub_height, sub_width, height))
    lower_right = image.crop((sub_width, sub_height, width, height))

    # 返回分割后的四个子图
    return upper_left, upper_right, lower_left, lower_right
```

```
# 调用函数进行图片分割
image_path = "potholes"
sub_images = split_image(image_path)

# 保存分割后的四个子图
sub_images[0].save("upper_left.jpg")
sub_images[1].save("upper_right.jpg")
sub_images[2].save("lower_left.jpg")
sub_images[3].save("lower_right.jpg")
```

附录 4：vgg16.py

VGG16 模型提取图像特征

```
import os
import torch
from torchvision import models, transforms
from PIL import Image
import numpy as np
from sklearn.decomposition import PCA
import pandas as pd

# 使用预训练的 VGG16 模型
base_model = models.vgg16(pretrained=True).features

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
base_model = base_model.to(device)

# 定义图像的预处理步骤
preprocess = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

def extract_features(img_path):
    """
    从给定图像路径提取特征
    """
    img = Image.open(img_path).convert("RGB")
    img_tensor = preprocess(img).unsqueeze(0).to(device)

    with torch.no_grad():
```

```
            features = base_model(img_tensor)

    flattened_features = features.view(features.size(0), -1).cpu().numpy()
    return flattened_features[0]

def extract_features_from_directory(directory):
    """
    从指定目录中的所有图像提取特征
    """
    features_list = []
    for filename in os.listdir(directory):
        if filename.endswith(('.jpg', '.png', '.jpeg')):  # 您可以根据需要添加其他格式
            img_path = os.path.join(directory, filename)
            features_list.append(extract_features(img_path))
    return features_list

# 提取特征
normal_features = extract_features_from_directory('./data/normal')
potholes_features = extract_features_from_directory('./data/potholes')

normal_labels = ['normal'] * len(normal_features)
potholes_labels = ['potholes'] * len(potholes_features)

# 合并特征和标签
all_features = normal_features + potholes_features
all_labels = normal_labels + potholes_labels

# 使用 PCA 进行降维
n_components = 100  # 例如，降维到 500 维
pca = PCA(n_components=n_components)
reduced_features = pca.fit_transform(all_features)

# 保存降维后的特征到 Excel 文件
def save_reduced_features_to_excel(features, labels, file_name):
    """
    将降维后的特征保存到 Excel 文件中
    """
    # 将特征列表转换为 Pandas DataFrame
    df = pd.DataFrame(features)

    # 添加标签列
    df['label'] = labels

    # 将 DataFrame 保存到 Excel 文件中
```

```
    with pd.ExcelWriter(file_name, engine='openpyxl') as writer:
        df.to_excel(writer, index=False)

save_reduced_features_to_excel(reduced_features, all_labels, 'reduced_features.xlsx')
```

附录 5：生成对抗样本.py

基于遗传算法生成对抗样本

```
import numpy as np
# from keras.preprocessing import image
# from keras.models import load_model
from PIL import Image

# # 加载预训练模型
# model = load_model('path_to_your_model.h5')

# Load the image and convert it to a NumPy array
def load_image(image_path):
    with Image.open(image_path) as img:
        return np.array(img)

# Save the image from a NumPy array
def save_image(array, image_path):
    img = Image.fromarray(np.uint8(array))
    img.save(image_path)

# Evaluation function based on brightness
def evaluate_brightness(image_data):
    brightness = np.mean(image_data)
    # if brightness < 255:
    #     return 1
    # else:
    #     return 0
    return -brightness**2 + 255*brightness

# 初始化种群
def initialize_population(img, population_size, mutation_rate):
    population = []
    for _ in range(population_size):
        # 随机扰动图像
        noise = np.random.randint(0, mutation_rate, img.shape, dtype=np.uint8)
        noisy_image = (img + noise) % 255
```

```
            population.append(noisy_image)
    return population

# 选择过程
def select(population, fitness_scores):
    # 这里我们简单地选择最好的两个
    sorted_population = [x for _, x in sorted(zip(fitness_scores, population), key=lambda
pair: pair[0])]
    return sorted_population[:2]

# 交叉和变异
def crossover_and_mutate(parent1, parent2, mutation_rate):
    child = parent1.copy()
    rows, cols, ch = parent1.shape
    # 随机选择交叉点
    crossover_point = np.random.randint(1, cols - 1)
    child[:, crossover_point:] = parent2[:, crossover_point:]
    # 变异
    mutation = np.random.randint(0, mutation_rate, child.shape, dtype=np.uint8)
    child = (child + mutation) % 256
    return child

# 加载原始图像
original_image = load_image('normal1.jpg')
target_label = 0  # 假设我们的目标是将这张图片分类为 0

# 遗传算法参数
population_size = 100
mutation_rate = 1  # 噪声强度
generations = 100 # 迭代次数

# 初始化种群
population = initialize_population(original_image, population_size, mutation_rate)

# 遗传算法主循环
for _ in range(generations):
    # Evaluate the population's brightness
    fitness_scores = [evaluate_brightness(member) for member in population]

    # Select the best individuals
    selected = select(population, fitness_scores)

    # 创建新一代
    next_generation = []
```

```
        for _ in range(int(population_size/2)):    # 创建足够的后代来填充种群
            child = crossover_and_mutate(selected[0], selected[1], mutation_rate)
            next_generation.append(child)

        # 新一代替换旧一代
        population = next_generation

# 选择最优个体
best_individual = population[0]
best_individual_fitness = evaluate_brightness(best_individual)
# 显示最优个体
Image.fromarray(best_individual.astype('uint8')).show()
print(f"Best individual fitness: {best_individual_fitness}")

best_image = Image.fromarray(best_individual.astype('uint8'))
best_image.save('adversarial_sample.png')
```

附录 6：switch_xml.py

图像文件格式转换 xml 代码

```
import xml.etree.ElementTree as ET
import os
from os import getcwd

sets = ['train', 'val', 'test']
classes = ["potholes"]    # 改成自己的类别
abs_path = os.getcwd()
print(abs_path)


def convert(size, box):
        dw = 1. / (size[0])
        dh = 1. / (size[1])
        x = (box[0] + box[1]) / 2.0 - 1
        y = (box[2] + box[3]) / 2.0 - 1
        w = box[1] - box[0]
        h = box[3] - box[2]
        x = x * dw
        w = w * dw
        y = y * dh
        h = h * dh
        return x, y, w, h
```

```
文件写入
def convert_annotation(image_id):
    in_file = open('D:\Desktop\yolov5-master\data\mydata\dataset\sn\%s.xml' % (image_id), encoding='UTF-8')#
    out_file = open('D:\Desktop\yolov5-master\data\mydata\dataset\label\%s.txt' % (image_id), 'w')
    tree = ET.parse(in_file)
    root = tree.getroot()
    size = root.find('size')
    w = int(size.find('width').text)
    h = int(size.find('height').text)
    for obj in root.iter('object'):
        difficult = obj.find('difficult').text
        # difficult = obj.find('Difficult').text
        cls = obj.find('name').text
        if cls not in classes or int(difficult) == 1:
            continue
        cls_id = classes.index(cls)
        xmlbox = obj.find('bndbox')
        b = (float(xmlbox.find('xmin').text), float(xmlbox.find('xmax').text), float(xmlbox.find('ymin').text),
             float(xmlbox.find('ymax').text))
        b1, b2, b3, b4 = b
        # 标注越界修正
        if b2 > w:
            b2 = w
        if b4 > h:
            b4 = h
        b = (b1, b2, b3, b4)
        bb = convert((w, h), b)
        out_file.write(str(cls_id) + " " + " ".join([str(a) for a in bb]) + '\n')


wd = getcwd()
for image_set in sets:
    if not os.path.exists('D:\Desktop\yolov5-master\data\mydata\dataset\label'):
        os.makedirs('D:\Desktop\yolov5-master\data\mydata\dataset\label')
    image_ids = open('D:\Desktop\yolov5-master\data\mydata\dataset\mai\%s.txt' % (image_set)).read().strip().split()

    if not os.path.exists('D:\Desktop\yolov5-master\data\mydata\dataset\pat'):
        os.makedirs('D:\Desktop\yolov5-master\data\mydata\dataset\pat')
```

```
    list_file = open('data\mydata\dataset\pat\%s.txt' % (image_set), 'w')
    # 这行路径不需更改，这是相对路径
    for image_id in image_ids:
        list_file.write('D:\Desktop\yolov5-master\data\mydata\dataset\data\%s.jpg\n'    %
(image_id))
        convert_annotation(image_id)
    list_file.close()
```

附录 7：split_train_val.py

训练验证集划分代码

```
import os
import random
import argparse

parser = argparse.ArgumentParser()
#xml 文件的地址，根据自己的数据进行修改 xml 一般存放在 Annotations 下
parser.add_argument('--xml_path',default='D:\Desktop\yolov5-
master\data\mydata\dataset\sn', type=str, help='input xml label path')
#数据集的划分，地址选择自己数据下的 ImageSets/Main
parser.add_argument('--txt_path',default='D:\Desktop\yolov5-
master\data\mydata\dataset\mai',type=str, help='output txt label path')
opt = parser.parse_args()

定义训练验证集所占比，这里没有测试集
trainval_percent = 1.0
训练集所占比重
train_percent = 0.8
xmlfilepath = opt.xml_path
txtsavepath = opt.txt_path
total_xml = os.listdir(xmlfilepath)
if not os.path.exists(txtsavepath):
    os.makedirs(txtsavepath)

num = len(total_xml)
list_index = range(num)
tv = int(num * trainval_percent)
tr = int(tv * train_percent)
trainval = random.sample(list_index, tv)
train = random.sample(trainval, tr)
写入相关数据进入文件
file_trainval = open(txtsavepath + '/trainval.txt', 'w')
```

```
file_test = open(txtsavepath + '/test.txt', 'w')
file_train = open(txtsavepath + '/train.txt', 'w')
file_val = open(txtsavepath + '/val.txt', 'w')

for i in list_index:
        name = total_xml[i][:-4] + '\n'
        if i in trainval:
                file_trainval.write(name)
                if i in train:
                        file_train.write(name)
                else:
                        file_val.write(name)
        else:
                file_test.write(name)

file_trainval.close()
file_train.close()
file_val.close()
file_test.close()
```